



24

Showing Progress

This chapter is all about communicating progress to the user.

React Native has different components that are used to handle the different types of progress that you want to communicate.

First, you'll learn why you need to communicate progress in the app. Then, you'll learn how to implement progress indicators and progress bars. And finally, you'll see specific examples that show you how to use progress indicators with navigation while data loads and progress bars to communicate the current position in a series of steps.

The following sections are covered in this chapter:

- Understanding progress and usability
- Indicating progress
- Measuring progress
- Exploring navigation indicators
- Step progress

Technical requirements

You can find the code files for this chapter on GitHub at

<https://github.com/PacktPublishing/React-and-React-Native-5E/tree/main/Chapter24>.

Understanding progress and usability

Imagine that you have a microwave oven that has no window and makes no sound. The only way to interact with it is by pressing a button labeled “cook.” As absurd as this device sounds, it’s what many software users face: no indication of progress. Is the microwave cooking anything? If so, how do we know when it will be done?

One way to improve the microwave situation is to add a beep sound. This way, the user gets feedback after pressing the cook button. You’ve overcome one hurdle, but the user is still left asking, “When will my food be ready?” Before you go out of business, you had better add some sort of progress measurement display, such as a timer.

It’s not that UI programmers don’t understand the basic principles of this usability concern; it’s just that they have stuff to do, and this sort of thing simply slips through the cracks in terms of priority. In React Native, there are components to give the

user indeterminate progress feedback and precise progress measurements. It's always a good idea to make these things a top priority if you want a good user experience.

Now that you understand the role of progress in usability, it's time to learn how to indicate progress in your React Native UIs.

Indicating progress

In this section, you'll learn how to use the `ActivityIndicator` component. As its name suggests, you render this component when you need to indicate to the user that something is happening. The actual progress may be indeterminate, but at least you have a standardized way to show that something is happening, despite there being no results to display yet.

Let's create an example so that you can see what this component looks like. Here's the `App` component:

```
import React from "react";
import { View, ActivityIndicator } from "react-native";
import styles from "./styles";
export default function App() {
  return (
    <View style={styles.container}>
      <ActivityIndicator size="large" />
    </View>
  );
}
```

```
);  
}
```

The `<ActivityIndicator />` component is platform-agnostic.

Here's how it looks on iOS:

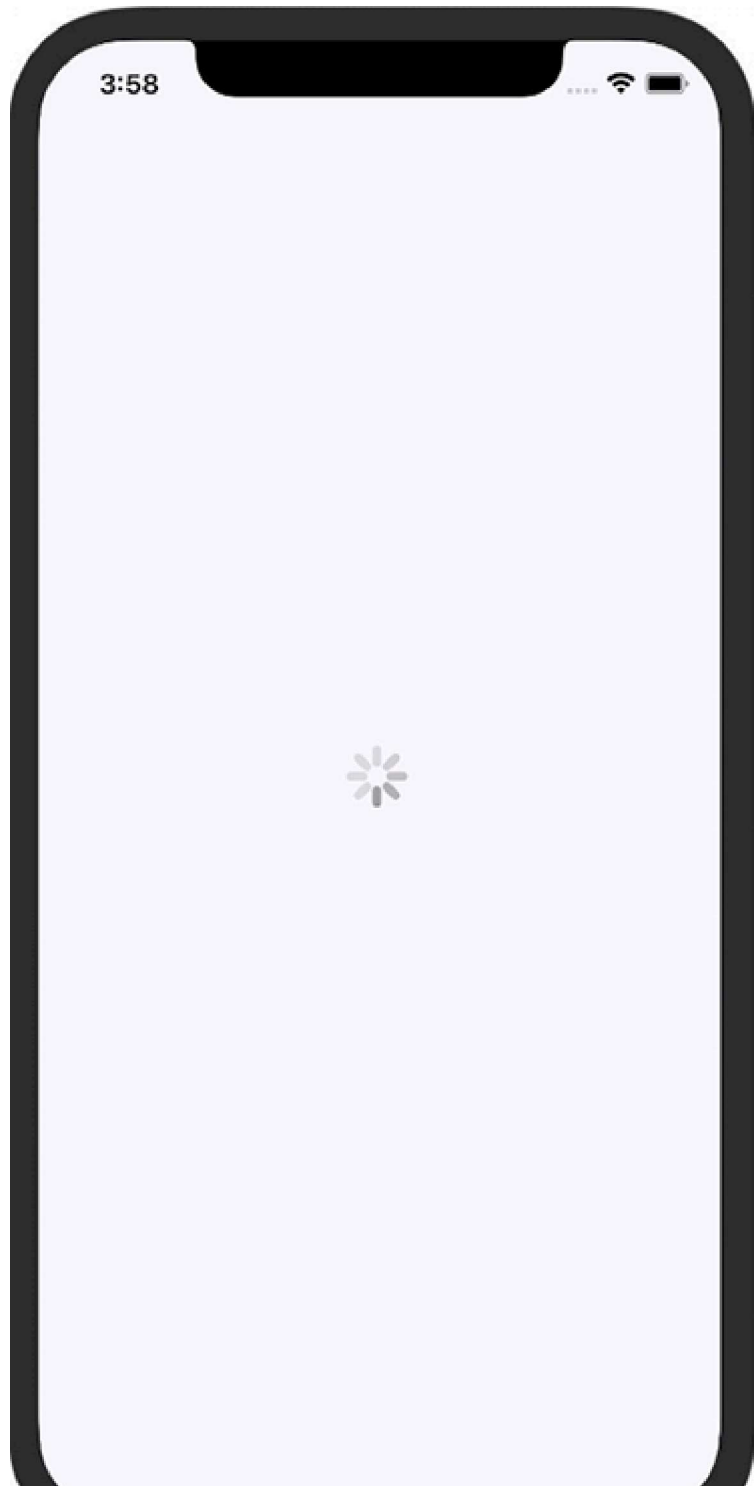




Figure 24.1: An activity indicator on iOS

It renders an animated spinner in the middle of the screen.

This is the large spinner, as specified in the `size` property. The `ActivityIndicator` spinner can also be small, which makes more sense if you're rendering it inside another smaller element.

Now, let's take a look at how this looks on an Android device:

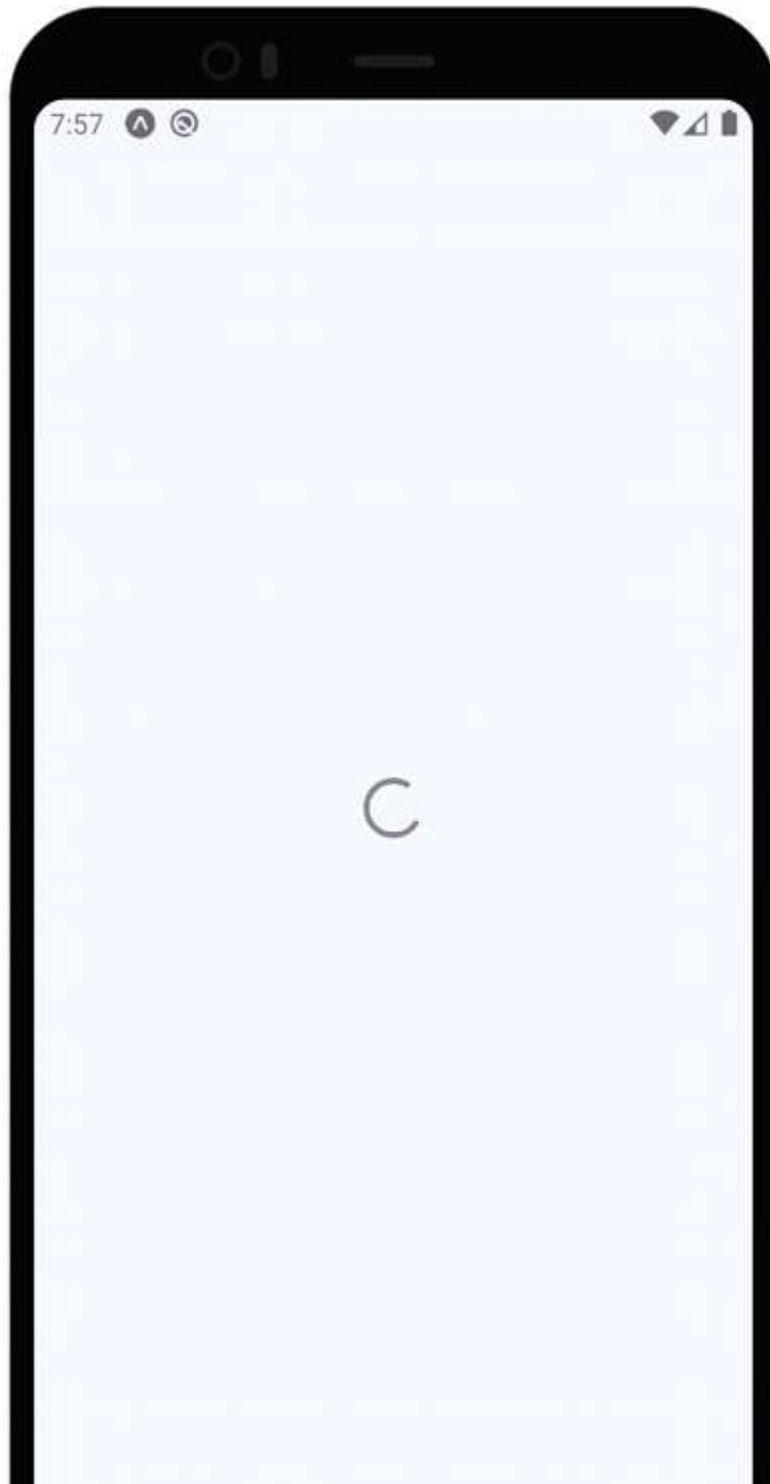




Figure 24.2: An activity indicator on Android

The spinner looks different, as it should, but your app conveys the same thing on both platforms: you're waiting for something.

This example spins forever. But don't worry: there's a more realistic progress indicator example coming up that shows you how to work with navigation and loading API data.

Exploring navigation indicators

Earlier in this chapter, you were introduced to the `ActivityIndicator` component. In this section, you'll learn how it can be used when navigating an application that loads data. For example, the user navigates from page or screen one to page two. However, page two needs to fetch data from the API that it can display to the user. So, while this network call is happening, it makes more sense to display a progress indicator instead of a screen devoid of useful information.

Doing this is actually kind of tricky because you have to make sure that the data that's required by the screen is fetched from the API each time the user navigates to the screen. Your goals should be as follows:

- Have the `Navigator` component automatically fetch API data for the scene that's about to be rendered.
- Use the promise that's returned by the API call as a means to display the spinner and hide it once the promise has been resolved.

Since your components probably don't care about whether a spinner is displayed or not, let's implement this as a generic `Wrapper` component:

```
export function LoadingWrapper({ children }: Props) {  
  const [loading, setLoading] = useState(true);  
  useEffect(() => {  
    setTimeout(() => {  
      setLoading(false);  
    }, 1000);  
  }, []);  
  if (loading) {  
    return (  
      <View style={styles.container}>  
        <ActivityIndicator size="large" />  
      </View>  
    );  
  }  
}
```

```
    } else {  
      return children;  
    }  
  }  
}
```

This `LoadingWrapper` component takes a `children` component and returns it (read renders) under a `loading` condition. It has a `useEffect()` hook with a timeout, and when it resolves, it changes the `loading` state to `false`. As you can see, the `loading` state determines whether the spinner or the `children` component is rendered.

With the `LoadingWrapper` component in place, let's take a look at the first screen component that you'll use with `react-navigation`:

```
const First = ({ navigation }: Props) => (  
  <LoadingWrapper>  
    <View style={styles.container}>  
      <Button title="Second" onPress={() => navigation.navigate("Second")} />  
      <Button title="Third" onPress={() => navigation.navigate("Third")} />  
    </View>  
  </LoadingWrapper>  
);
```

This component renders a layout that's wrapped with the `LoadingWrapper` component we created earlier. It wraps the

whole screen so that a spinner is displayed while the `setTimeout` method is pending. This is a useful approach to hiding extra logic in one place and reusing it on every page. Instead of the `setTimeout` method, in a real app, you can pass additional props to the `LoadingWrapper` and have full control of `loading` state from that screen itself.

Measuring progress

The downside of just indicating that progress is being made is that there's no end in sight for the user. This leads to a feeling of unease, like when you're waiting for food to cook in a microwave with no timer. When you know how much progress has been made and how much is left to go, you feel better. That is why it's always better to use a deterministic progress bar whenever possible.

Unlike the `ActivityIndicator` component, there's no platform-agnostic component in React Native for progress bars. So, we'll use the `react-native-progress` library for rendering progress bars.



In the past, React-Native had special components for showing progress bars for iOS and Android, but due to React-Native size optimization, the Meta team is working on moving such components to separate packages. So, `ProgressViewIOS`

and `ProgressBarAndroid` have been moved outside of the React-Native library.

Now, let's build the `ProgressBar` component that the application will use:

```
import * as Progress from "react-native-progress";
type ProgressBarProps = {
  progress: number;
};
export default function ProgressBar({ progress }: ProgressBarProps) {
  return (
    <View style={styles.progress}>
      <Text style={styles.progressText}>{Math.round(progress * 100)}%</Text>
      <Progress.Bar width={200} useNativeDriver progress={progress} />
    </View>
  );
}
```

The `ProgressBar` component accepts the `progress` property and renders the label and progress bar. The `<Progress.Bar />` component accepts a set of props, but we need only `width`, `progress`, and `useNativeDriver` (for better animation). Now, let's put this component to use in the `App` component:

```
export default function MeasuringProgress() {
  const [progress, setProgress] = useState(0);
```

```
useEffect(() => {
  let timeoutRef: NodeJS.Timeout | null = null;
  function updateProgress() {
    setProgress((currentProgress) => {
      if (currentProgress < 1) {
        return currentProgress + 0.01;
      } else {
        return 0;
      }
    });
  }
  timeoutRef = setTimeout(updateProgress, 100);
}
updateProgress();
return () => {
  timeoutRef && clearTimeout(timeoutRef);
};
}, []);
return (
  <View style={styles.container}>
    <ProgressBar progress={progress} />
  </View>
);
}
```

Initially, the `<ProgressBar>` component is rendered at 0%. In the `useEffect()` hook, the `updateProgress()` function uses a timer to simulate a real process that you want to show the progress of.



In the real world, you'll probably never use a simulation of timers. However, there are specific scenarios where this approach might be valuable, such as when displaying statistical data or monitoring the progress of file uploads to servers. In these situations, even though you're not relying on a direct timer, you will still have access to a current progress value that you can use.

Here's what the screen looks like:

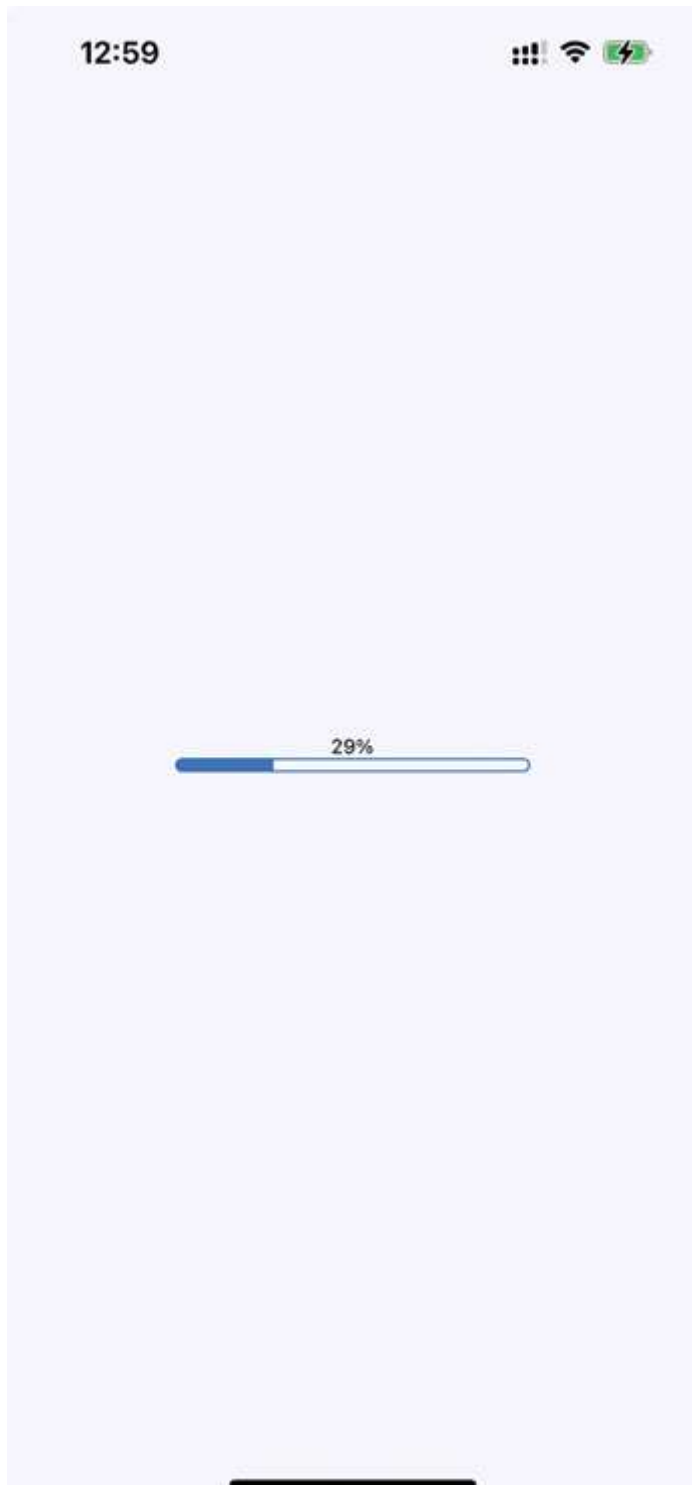


Figure 24.3: The progress bar

Showing a quantitative measure of progress is important so that users can gauge how long something will take. In the next section, you'll learn how to use step progress bars to show the user where they are in terms of navigating screens.

Step progress

In this final example, you'll build an app that displays the user's progress through a predefined number of steps. For example, it might make sense to split a form into several logical sections and organize them in such a way that, as the user completes one section, they move to the next step. A progress bar would be helpful feedback for the user.

You'll insert a progress bar into the navigation bar, just below the title, so that the user knows how far they've gone and how far is left to go. You'll also reuse the `ProgressBar` component that you used earlier in this chapter.

Let's take a look at the result first. There are four screens in this app that the user can navigate. Here's what the **First** page (scene) looks like:

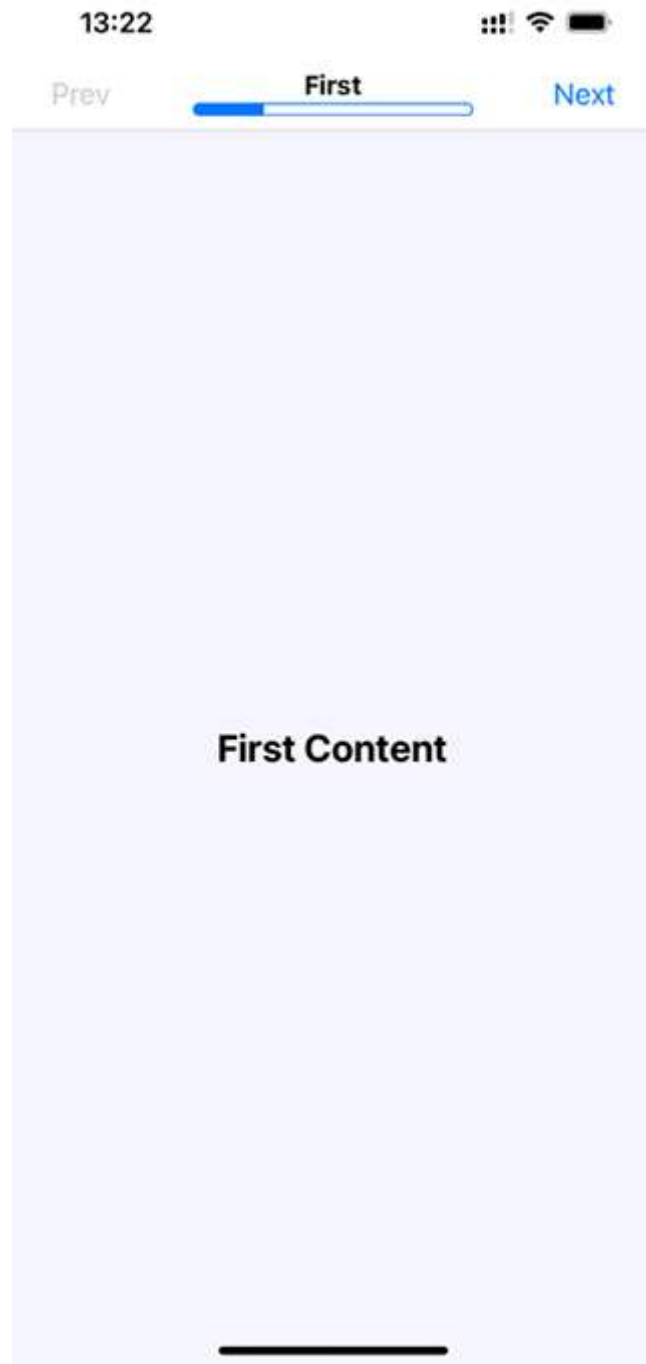


Figure 24.4: The first screen

The progress bar under the title reflects the fact that the user is 25% through the navigation. Let's see what the **Third** screen looks like:

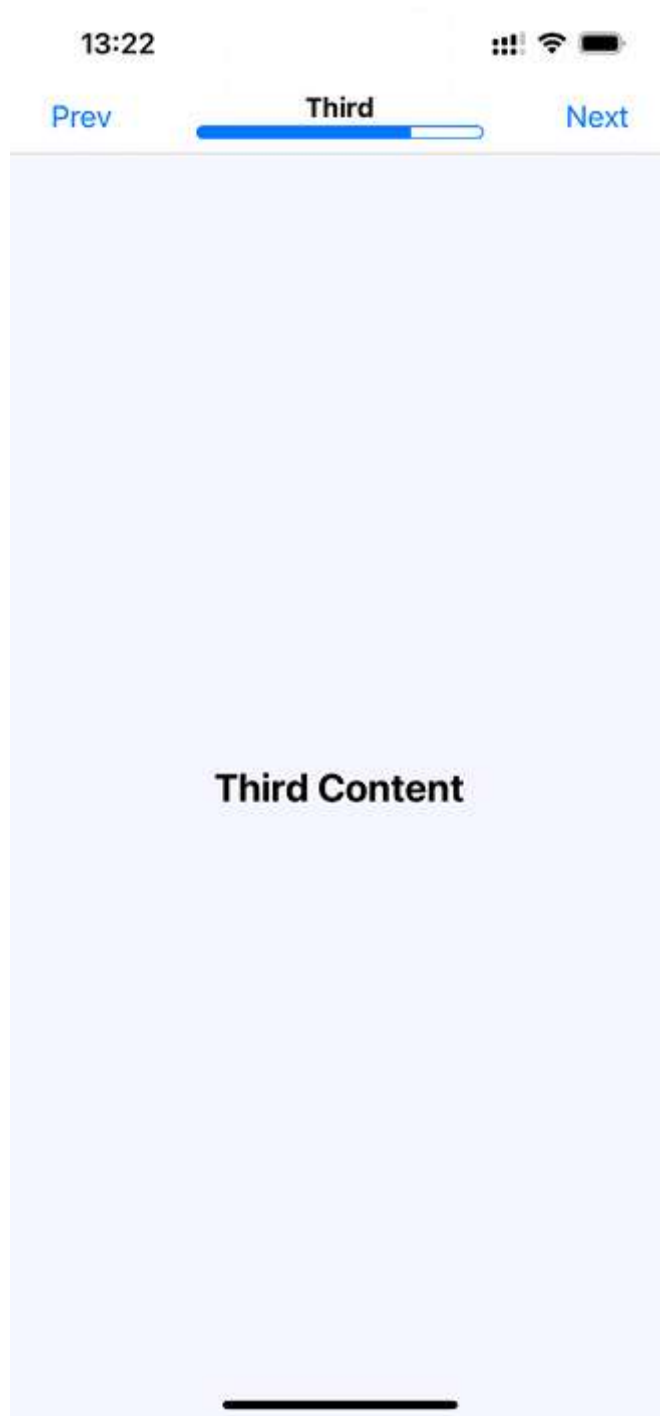


Figure 24.5: The third screen

The progress is updated to reflect where the user is in the route stack. Let's take a look at the `App` component here:

<https://github.com/PacktPublishing/React-and-React-Native-5E/blob/main/Chapter21/step-progress-new/App.tsx>.

This app has four screens. The components that render each of these screens are stored in the `routes` constant, which is then used to configure the stack navigator using `createNativeStackNavigator()`. The reason for creating the `routes` array is so that it can be used by the `progress` parameter that is passed by `initialParams` to every route. To calculate the progress, we take the current route index as a value of the route's length.

For example, `Second` is in the number 2 position (an index of 1 + 1) and the length of the array is 4. This will set the progress bar to 50%.

Also, the **Next** and **Previous** buttons' calls to `navigation.navigate()` have to pass `routeName`, so we added the `nextRouteName` and `prevRouteName` variables to the `screenOptions` handler.

Summary

In this chapter, you learned how to show your users that something is happening behind the scenes. First, we discussed why showing progress is important for the usability of an application. Then, we implemented a basic screen that indicated progress was being made. After that, we implemented a `ProgressBar` component, which is used to measure specific progress amounts.

Indicators are good for indeterminate progress. We implemented navigation that showed progress indicators while network calls were pending. In the final section, we implemented a progress bar that showed the user where they were in a predefined number of steps.

In the next chapter, we'll look at React Native maps and geolocation data in action.